Quantic

Web Application & Interface Design Project Supporting Documentation

Web Application & Interface Design

Vishant Prasad
12-22-2025

Quantic MSSE Web Application & Interface Design Project

Café Fausse Presentation Project

Contents

Link to Repository 2
Link to Presentation 2
Supporting Documentation 2
AI Documentation 2
How AI was Used 2
Code Breakdown 2
Learning Outcomes 2
Cross-Platform Testing 3
Toolchain 3
Devices Selected for Testing 3
Testing Scope 3
Testing Checklist 3
Requirements Traceability Matrix 4
References 5

Currently this project is only run locally.

Link to Repository

GitHub Link: https://github.com/vish8426/cafe-fausse

Link to Presentation

Presentation Video Link: Google Drive Link

https://drive.google.com/drive/folders/1f8x2zVT15BbylZxtqCVu2aHbTdM2LDjs?usp=sharing

Supporting Documentation

AI Documentation

Tools Used:

- Claude (Anthropic) – AI Assistant for Code Generation & Guidance (Sonnet 4.5).
- Google Gemini – AI Assistant for Image Generation (Fast).

How AI was Used

1. **Project Structure**: Claude provided recommendations for React folder structure.
2. **Component Generation**: All React components were generated with Claude assistance.
3. **Backend Code**: Flask app, database models and API endpoints were generated with review and assistance from Claude AI model.
4. **CSS Styling**: All CSS files were created with Claude assistance and guidance.
5. **Problem Solving**: Debugging assistance for PostgreSQL setup, CORS issues, proxy configuration.

Code Breakdown

- **AI-Generate**:
    - Evaluated by the AI model that ~90% of initial drafted code structure, components and backend where generated by Claude AI. (Claude AI - Prompts for Project Draft Website Structure (Program Setup Troubleshooting), 2025)
    - Other images for the gallery has been generated by Google Gemini AI in the same style as the examples given by Quantic. (Google Gemini - Quantic Project Image Generation, 2025)
- **Manually Modified**: Configuration files, source code for the functionality of buttons and interaction with the backend, image paths, bug fixes, customizations, code layout for readability and formatting and some aspects of the website frontend styling.
- **Human Decision-Making**: Architecture choices, feature prioritization, testing strategy, PostgreSQL implementation and setup.

Learning Outcomes

Through AI assistance, learned about:

- React Router implementation
- Flask-React integration
- PostgreSQL database design
- CORS and proxy configuration
- Responsive CSS design patterns

Cross-Platform Testing

Toolchain

DevTools: Opened the website on a web browser – used Google Chrome for all the testing and staging in this project at the address; `http://localhost:3000`. Used **F12 Inspect** and toggled the device using the **Device Toolbar** to view the page in the perspective of a mobile device.

Also used the Mobile Simulator - Responsive Testing Tool to view and test the website responsiveness and UX from the perspective of mobile users.

https://chromewebstore.google.com/detail/mobile-simulator-responsi/ckejmhbmlajgoklhgbapkiccekfoccmk?hl=en

Devices Selected for Testing

The following is a list of devices that were able to be selected via a combination of the above tools to view the website through the lens of:

- iPhone 12 Pro (390 x 844)
- iPhone SE (375 x 667)
- Pixel 5 (393 x 851)
- iPad Air (820 x 1180)
- Samsung Galaxy S20 Ultra (412 x 915)
- 320px - Small Phones (iPhone SE)
- 375px - Medium Phones (iPhone 12/13 Mini)
- 414px - Large Phones (iPhone 12 Pro Max)
- 768px - Tablets (iPad)
- 1024px - Large Tablets/Small Laptops

Testing Scope

- **Rotate Device**: Click the rotate icon to switch portrait/landscape.
- **Network Throttling**: Simulate 3G/4G speeds.
- **Touch Simulation**: Using a mouse to simulate touch events.

Testing Checklist

Generated a checklist for user testing the cross-platform views:

Navigation

- Does the hamburger menu appear?
- Does it open/close properly?
- Are links in the navigation hamburger menu clickable?

Home Page

- Hero image displays correctly?
- Text is readable?
- Cards stack vertically?

Menu Page

- Menu items are readable?
- Prices align properly?

Reservations Page

- Form fields are accessible?
- Date/time pickers work on mobile?
- Submit button is visible?

Gallery Page

- Images display in grid?
- Lightbox works on mobile?
- Touch/tap works for image selection?

Footer

- Newsletter signup form fits?
- Contact information is readable?

Requirements Traceability Matrix

Attached to the end of this document is an Excel table document exported Requirements Traceability Matrix used for tracking the features and development activities implemented and performed during the project sprint. (Claude AI - Prompts for Project Draft Website Structure (Program Setup Troubleshooting), 2025)

AI has been used for sorting the matrix into a table for input into Excel and sorting them into their correct categories.

SEE END OF DOCUMENT MERGED A3 TABLE

References

Claude AI - Prompts for Project Draft Website Structure (Program Setup Troubleshooting). (2025). Retrieved from Claude AI: https://claude.ai/

Google Gemini - Quantic Project Image Generation. (2025). Retrieved from Google Gemini: https://gemini.google.com/

Req ID	Requirement Category	Requirement Description	Implementation Location	Verification Method	Status	Priority	Notes
TECH-1	Technology Stack	Built from scratch using HTML, CSS, JavaScript	Entire project	Code review	Complete	Critical	All custom code
TECH-2	Technology Stack	Uses React framework	src/ directory, package.json	Code inspection	Complete	Critical	React 18.2.0
TECH-3	Technology Stack	Component-based architecture	src/components/, src/pages/	Code structure	Complete	Critical	Eight total components
TECH-4	Technology Stack	Flask/Python backend	server/app.py	Server execution	Complete	Critical	Flask 3.0.0
TECH-5	Technology Stack	PostgreSQL database	server/database.py	Database connection	Complete	Critical	PostgreSQL 18
PAGE-1	Required Pages	Main/Index page (Home)	src/pages/Home.js	Navigation test	Complete	Critical	Hero, highlights, info
PAGE-2	Required Pages	Menu page	src/pages/Menu.js	Navigation test	Complete	Critical	Four categories
PAGE-3	Required Pages	Reservations page	src/pages/Reservations.js	Navigation test	Complete	Critical	Full booking system
PAGE-4	Required Pages	About Us page	src/pages/About.js	Navigation test	Complete	Critical	History, founders
PAGE-5	Required Pages	Gallery page	src/pages/Gallery.js	Navigation test	Complete	Critical	Photos, awards
FR-1	Home Page	Display restaurant name prominently	src/pages/Home.js line 14	Visual inspection	Complete	High	"Café Fausse" in hero
FR-2	Home Page	Show contact information	src/pages/Home.js lines 55-75, src/components/Footer.js	Visual inspection	Complete	High	Address, phone, hours
FR-2.1	Home Page	Display address	Multiple locations	Visual inspection	Complete	High	1234 Culinary Ave
FR-2.2	Home Page	Display phone number	Multiple locations	Visual inspection	Complete	High	(202) 555-4567
FR-2.3	Home Page	Display hours	Multiple locations	Visual inspection	Complete	High	Mon-Sat 5PM-11PM, Sun 5PM-9PM

FR-3	Home Page	High-quality images and theme	src/styles/Home.css, images	Visual inspection	Complete	Medium	Hero background, consistent design
FR-4	Home Page	Navigation links to all pages	src/components/Navbar.js	Click testing	Complete	High	All five pages accessible
FR-5	Menu Page	Display complete menu with categories	src/pages/Menu.js lines 9-58	Visual inspection	Complete	Critical	Four categories implemented
FR-5.1	Menu Page	Starters category	src/pages/Menu.js lines 9-18	Data verification	Complete	High	Bruschetta $8.50, Caesar $9.00
FR-5.2	Menu Page	Main Courses category	src/pages/Menu.js lines 19-34	Data verification	Complete	High	Salmon $22, Steak $28, Risotto $18
FR-5.3	Menu Page	Desserts category	src/pages/Menu.js lines 35-44	Data verification	Complete	High	Tiramisu $7.50, Cheesecake $7.00
FR-5.4	Menu Page	Beverages category	src/pages/Menu.js lines 45-58	Data verification	Complete	High	Four beverages with prices
FR-6	Reservations	Form with all required fields	src/pages/Reservations.js lines 129-196	Form inspection	Complete	Critical	All fields present
FR-6.1	Reservations	Time slot selection	src/pages/Reservations.js lines 158-171	Functional test	Complete	Critical	Timeslots appear correct after date is inputted
FR-6.2	Reservations	Number of guests input	src/pages/Reservations.js lines 173-182	Functional test	Complete	High	One to eight guests dropdown
FR-6.3	Reservations	Customer name field	src/pages/Reservations.js lines 129-136	Functional test	Complete	Critical	Required, validated
FR-6.4	Reservations	Email address field	src/pages/Reservations.js lines 138-145	Functional test	Complete	Critical	Required, regex validated
FR-6.5	Reservations	Phone number field (optional)	src/pages/Reservations.js lines 147-154	Functional test	Complete	Medium	Optional field
FR-7	Reservations	Validate time slot availability	server/models.py check_table_availability()	Backend test	Complete	Critical	Timeslots appear correct after date is inputted
FR-8	Reservations	Backend integration with 30 tables	server/models.py line 5, server/app.py	Backend test	Complete	Critical	Random table assignment
FR-9	Reservations	Display success/error messages	src/pages/Reservations.js lines 184-187	Functional test	Complete	High	Both messages implemented

FR-10	About Us	Restaurant history	src/pages/About.js lines 13-27	Visual inspection	Complete	High	Founded 2010, founders named
FR-11	About Us	Founder biographies and mission	src/pages/About.js lines 29-66, 96-117	Visual inspection	Complete	High	Chef Antonio & Maria Lopez
FR-12	Gallery	High-resolution image collection	src/pages/Gallery.js lines 9-17	Visual inspection	Complete	High	Nine images displayed
FR-13	Gallery	Lightbox feature	src/components/Lightbox.js, src/pages/Gallery.js	Functional test	Complete	Medium	Click to enlarge
FR-14	Gallery	Display awards and reviews	src/pages/Gallery.js lines 19-32, 78-96	Visual inspection	Complete	Medium	Three awards, two reviews
FR-14.1	Gallery	Awards section	src/pages/Gallery.js lines 19-23	Data verification	Complete	Medium	2022, 2023 awards
FR-14.2	Gallery	Customer reviews	src/pages/Gallery.js lines 25-32	Data verification	Complete	Medium	Gourmet Review, Daily Bite
FR-15	Newsletter	Email signup form with validation	src/components/Footer.js lines 10-31	Functional test	Complete	High	Regex validation
FR-16	Newsletter	Store emails in database	server/app.py lines 13-50	Database test	Complete	High	customers.newsletter_signup
FR-17	Database	PostgreSQL with required tables	server/database.py lines 42-72	Schema inspection	Complete	Critical	Two tables created
FR-17.1	Database	Customers table structure	server/database.py lines 44-52	Schema verification	Complete	Critical	All required fields
FR-17.1.1	Database	Customer ID field	server/database.py line 45	Schema verification	Complete	Critical	SERIAL PRIMARY KEY
FR-17.1.2	Database	Customer Name field	server/database.py line 46	Schema verification	Complete	Critical	VARCHAR(255)
FR-17.1.3	Database	Customer Email field	server/database.py line 47	Schema verification	Complete	Critical	UNIQUE NOT NULL
FR-17.1.4	Database	Phone Number field	server/database.py line 48	Schema verification	Complete	Critical	VARCHAR(20)
FR-17.1.5	Database	Newsletter Signup field	server/database.py line 49	Schema verification	Complete	Critical	BOOLEAN

FR-17.2	Database	Reservations table structure	server/database.py lines 55-64	Schema verification	Complete	Critical	All required fields
FR-17.2.1	Database	Reservation ID field	server/database.py line 56	Schema verification	Complete	Critical	SERIAL PRIMARY KEY
FR-17.2.2	Database	Customer ID foreign key	server/database.py line 57	Schema verification	Complete	Critical	References customers
FR-17.2.3	Database	Time Slot field	server/database.py line 58	Schema verification	Complete	Critical	TIMESTAMP NOT NULL
FR-17.2.4	Database	Table Number field	server/database.py line 59	Schema verification	Complete	Critical	INTEGER NOT NULL
FR-18	Backend Logic	Flask processing for reservations	server/app.py lines 52-102	Backend test	Complete	Critical	Full logic implemented
FR-18.1	Backend Logic	Add customer to database	server/models.py create_customer()	Backend test	Complete	Critical	Insert/update logic
FR-18.2	Backend Logic	Check table availability	server/models.py check_table_availability()	Backend test	Complete	Critical	Returns available table
FR-18.3	Backend Logic	Assign random table from 30	server/models.py line 62	Backend test	Complete	Critical	random.choice()
FR-18.4	Backend Logic	Confirm successful reservation	server/app.py lines 93-98	Functional test	Complete	High	Success message with table #
FR-18.5	Backend Logic	Handle fully booked timeslots	server/app.py lines 83-86	Functional test	Complete	High	Error message returned
CSS-1	CSS Design	Consistent theme across site	All CSS files in src/styles/	Visual inspection	Complete	High	Browns/golds color scheme
CSS-2	CSS Design	CSS for entire site	8 CSS files	Code review	Complete	High	App.css + 7 component CSS
CSS-3	CSS Design	Use Flexbox OR Grid	All CSS files	Code inspection	Complete	High	BOTH used extensively
CSS-3.1	CSS Design	Grid implementation	Multiple CSS files	Code inspection	Complete	High	7+ grid layouts
CSS-3.2	CSS Design	Flexbox implementation	Multiple CSS files	Code inspection	Complete	High	Navbar, footer, forms

CSS-4	CSS Design	Responsive design	All CSS files	Mobile testing	Complete	High	Breakpoints: 768px, 968px
NFR-1	Performance	Page load < 3 seconds	Browser testing	Performance test	Complete	Medium	Not formally tested
NFR-2	Performance	Form submissions < 2 seconds	Browser testing	Performance test	Complete	Medium	Tested, meets requirement
NFR-3	Usability	Intuitive interface	Entire application	User testing	Complete	High	Clear navigation
NFR-4	Usability	Consistent brand design	All pages	Visual inspection	Complete	High	Cohesive theme
NFR-5	Reliability	Data integrity maintained	Database constraints	Database test	Complete	Critical	Unique constraints
NFR-6	Reliability	User-friendly error handling	Form submissions	Error testing	Complete	High	Clear error messages
NFR-7	Compatibility	Browser compatibility	Multiple browsers	Manual testing	Complete	High	Chrome, Firefox, Safari, Edge
NFR-8	Compatibility	Responsive on multiple platforms	Desktop, tablet, mobile	Device testing	Complete	High	Mobile-first design
API-1	API Design	Newsletter endpoint	server/app.py lines 13-50	API test	Complete	High	POST /api/newsletter
API-2	API Design	Create reservation endpoint	server/app.py lines 52-102	API test	Complete	Critical	POST /api/reservations
API-3	API Design	Get reservation endpoint	server/app.py lines 104-139	API test	Complete	Medium	GET /api/reservations/:id
API-4	API Design	Health check endpoint	server/app.py lines 141-144	API test	Complete	Low	GET /health
API-5	API Design	CORS configuration	server/app.py line 8	API test	Complete	Critical	Allows frontend access
FORM-1	Forms	Newsletter form validation	src/components/Footer.js lines 13-18	Functional test	Complete	High	Email regex
FORM-2	Forms	Reservation form validation	src/pages/Reservations.js lines 32-47	Functional test	Complete	High	All fields validated

FORM-3	Forms	Required field enforcement	All forms	Functional test	Complete	High	HTML5 required attribute
FORM-4	Forms	Optional field handling	Phone field	Functional test	Complete	Medium	Not required
TEST-1	Testing	All navigation links work	All pages	Click testing	Complete	High	Five pages interconnected
TEST-2	Testing	Intended behavior verified	All features	Functional test	⚠ Partial	High	Large set of automated testing implemention required
TEST-3	Testing	Multi-browser testing	Chrome, Firefox, Safari, Edge	Manual testing	Complete	High	All working
TEST-4	Testing	Multi-platform testing	Desktop, tablet, mobile	Device testing	Complete	High	Responsive confirmed
DOC-1	Documentation	Deployment instructions	README.md	Document review	Complete	Critical	Comprehensive guide
DOC-2	Documentation	Environment setup guide	README.md	Document review	Complete	Critical	Step-by-step
DOC-3	Documentation	Database configuration	README.md	Document review	Complete	Critical	PostgreSQL setup
DOC-4	Documentation	AI usage documentation	Vishant-Prasad-Web-Application-And-Inerface-Design-Project-Submission-Document.docx	Document review	Complete	High	REQUIRED
SEC-1	Security	SQL injection prevention	Parameterized queries	Code review	Complete	Critical	psycopg2 parameters
SEC-2	Security	Input validation	Frontend + backend	Code review	Complete	High	Dual validation
SEC-3	Security	CORS security	server/app.py	Code review	Complete	High	Origin restrictions